Conceptual Data Mapping: IDTA 02006-3-0 Digital Nameplate for Industrial Equipment ↔ OpenUSD (Bidirectional)

```
**{octicon}`tag;1em` Document Version:** {bdg-secondary}`0.1.0`
<br>**{octicon}`calendar;1em` Last Update:** {bdg-secondary}`2026-04-11`
```

Introduction

Overview

This document defines a **bidirectional** mapping between the **IDTA 02006-3-0 Digital Nameplate for Industrial Equipment** and OpenUSD. It covers both translation directions:

- **AAS → USD**: How AAS Submodel data is encoded in a USD prim using the `DppNameplateAPI` and `IndustrialEquipmentAPI` applied schemas, including schema definitions, encoding rules, and usage examples.
- **USD → AAS**: How USD prim data is read back and reconstructed into a conformant AAS `DigitalNameplate` Submodel instance, including reconstruction steps, fallback rules, and round-trip fidelity analysis.

IDTA 02006-3-0 is the foundational nameplate Submodel Template for industrial equipment — the base from which domain-specific templates such as the IDTA 02035-1 Digital Battery Passport Nameplate derive. This mapping is a concrete example supporting the **Identifier Separation of Concerns** proposal: the Digital Nameplate contains multiple identifier types with distinct scopes (product URI, order codes, article numbers, serial numbers, facility identifiers) that the proposal explicitly addresses.

Schema Architecture

The IDTA inheritance structure is mirrored directly in the USD schema composition:

```
DppNameplateAPI          (this document — full IDTA 02006-3-0 base, dpp: namespace)
  ├── + BatteryNameplateAPI     → battery passport (IDTA 02035-1)
  └── + IndustrialEquipmentAPI  → full industrial nameplate (IDTA 02006-3-0)
```

`DppNameplateAPI` is the shared base schema covering all IDTA 02006-3-0 fields. It is fully defined in this document; `dpp_openusd_bidirectional_mapping.md` documents the battery-passport-specific usage (which fields are mandatory for that context, plus `BatteryNameplateAPI`).

`IndustrialEquipmentAPI` is a lightweight extension schema applied to equipment prims that carry the full IDTA 02006-3-0 encoding, including the optional `Markings/ExplosionSafeties` nested structure and the `AssetSpecificProperties` child prim. It serves both as a discoverer (tooling can find all fully-encoded industrial nameplates by checking for `IndustrialEquipmentAPI`) and as the definitional home for `ExplosionSafetyAPI`.

```
**Dependency on the Source Identifier proposal.** Several mappings — in particular `URIOfTheProduct`, `OrderCodeOfManufacturer`, and `ProductArticleNumberOfManufacturer` — use a `sourceId:dpp:*` source identifier mechanism to store external identifiers verbatim alongside USD attributes. This mechanism is defined in the [Identifier Separation of Concerns proposal](./README.md) and is not yet part of the OpenUSD standard.
```

References

AAS / IDTA Reference

Version	Reference Documents
3.0 (December 2023)	IDTA 02006-3-0: Digital Nameplate for Industrial Equipment
1.0	IDTA 02002-1-0: Generic Frame for Technical Data for Industrial Equipment (Contact Information SMT)
—	IDTA-01001-3-1-2: Specification of the Asset Administration Shell Part 1: Metamodel (V3.1.2)

OpenUSD Reference

Version	Reference Documents
24.08	OpenUSD C++ and Schema Documentation, OpenUSD Github Repository
— (proposed)	Identifier Separation of Concerns — defines the `sourceId:*` mechanism used in this document

General Assumptions and Constraints

- **Two-way mapping** (AAS ↔ USD). Round-trip fidelity is a goal but not fully achievable; lossy fields are explicitly documented.
- The Digital Nameplate is encoded using `DppNameplateAPI` (all IDTA 02006-3-0 fields) and `IndustrialEquipmentAPI` (industrial-specific extensions). Schema definitions are in DigitalNameplate (Submodel).
- **ContactInformation** (IDTA 02002-1 drop-in, cardinality 1): two encoding options — flat `dpp:contact:*` attributes on the same prim (Option 1, simple cases) or a child `ContactInformation` scope prim (Option 2, complex/shared contact data). Only primary string values of MLP fields are stored in attributes; language variants go to `customData`.
- **Markings/ExplosionSafeties**: Each `Markings__NN__` SMC → child prim `Marking_NN` with `MarkingAPI` applied. The optional `ExplosionSafeties` SMC within a marking → nested `ExplosionSafeties` scope prim containing `ExplosionSafety_NN` child prims with `ExplosionSafetyAPI` applied.
- **AssetSpecificProperties**: Encoded as a child scope prim with `GuidelineProps_NN` children. See AssetSpecificProperties.
- **AAS structural context** (Asset, AssetAdministrationShell, Submodel containment) is not encoded in

USD. USD → AAS produces a Submodel instance only.

- **MLP fields**: USD stores only the primary string value. Language variants in `customData` survive round-trip only if USD tooling does not strip `customData`.
- **semanticIds** are preserved in `customData`. They survive round-trip only if `customData` is not stripped.
- **semanticId accuracy**: semanticIds listed here are derived from the IDTA 02006-3-0 specification. Verify against the PDF where marked as uncertain.

Definitions, Acronyms, Abbreviations

Term	Description
AAS	Asset Administration Shell
SMT	Submodel Template
SMC	SubmodelElementCollection
SML	SubmodelElementList
MLP	MultiLanguageProperty
IRDI	International Registration Data Identifier (IEC CDD/ECLASS)
`DppNameplateAPI`	Full IDTA 02006-3-0 base applied schema (`dpp:` prefix)
`IndustrialEquipmentAPI`	Industrial equipment extension schema; marker + `ExplosionSafetyAPI` home
`BatteryNameplateAPI`	Battery passport extension (IDTA 02035-1, see dpp_openusd_bidirectional_mapping.md)
`MarkingAPI`	Applied schema for a single Marking entry (shared across nameplate types)
`ExplosionSafetyAPI`	Applied schema for a single ExplosionSafety entry
`sourceId:dpp:*`	Source identifier attributes per the Identifier Separation of Concerns proposal

Concepts

High-Level Concept Mapping

AAS Element	OpenUSD	Round-trip	Notes
DigitalNameplate (Submodel)	Prim with `DppNameplateAPI` + `IndustrialEquipmentAPI`	Partial	AAS envelope not round-tripped
URIOfTheProduct	`sourceId:dpp:uriOfTheProduct` (string) + `dpp:uriOfTheProduct` (asset)	Lossless	Source id is primary
ManufacturerName	`dpp:manufacturerName` (string)	Lossy	Primary language only
ManufacturerProductDesignation	`dpp:manufacturerProductDesignation` (string)	Lossy	Mandatory (1)

AAS Element	OpenUSD	Round-trip	Notes
ContactInformation	`dpp:contact:*` attrs or child prim	Partial	Complex nesting; see field section
ManufacturerProductRoot	`dpp:manufacturerProductRoot` (string)	Lossy	Optional
ManufacturerProductFamily	`dpp:manufacturerProductFamily` (string)	Lossy	Optional
ManufacturerProductType	`dpp:manufacturerProductType` (string)	Lossy	Optional
OrderCodeOfManufacturer	`sourceId:dpp:orderCode` (string) + `dpp:orderCode` (string)	Lossy	Mandatory (1); source id recommended
ProductArticleNumberOfManufacturer	`sourceId:dpp:articleNumber` (string) + `dpp:articleNumber` (string)	Lossy	Optional; source id recommended
SerialNumber	`dpp:serialNumber` (string)	Lossless	Optional
BatchNumber	`dpp:batchNumber` (string)	Lossless	Optional; not in v3.0 template
CountryOfOrigin	`dpp:countryOfOrigin` (string)	Lossless	Optional; ISO 3166-1
YearOfConstruction	`dpp:yearOfConstruction` (string)	Lossless	Optional; YYYY
DateOfManufacture	`dpp:dateOfManufacture` (string)	Lossless	Optional; ISO 8601
UniqueFacilityIdentifier	`dpp:uniqueFacilityIdentifier` (string)	Lossless	Optional; v3.0 addition
HardwareVersion	`dpp:hardwareVersion` (string)	Lossy	Optional
FirmwareVersion	`dpp:firmwareVersion` (string)	Lossy	Optional
SoftwareVersion	`dpp:softwareVersion` (string)	Lossy	Optional
CompanyLogo	`dpp:companyLogo` (asset)	Lossless	Optional
Markings (SML)	Child prims under `Markings/` with `MarkingAPI`	Partial	`ExplosionSafeties` via nested child prims
AssetSpecificProperties (SMC)	Child scope prim `AssetSpecificProperties`	Partial	—
—	`assetInfo["version"]`	Dropped	No AAS equivalent
—	Composition arcs	Dropped	No AAS equivalent
—	Time-varying attributes	Dropped	No AAS equivalent
—	Variant sets	Dropped	No AAS equivalent
—	Relationships	Dropped	No AAS equivalent

DigitalNameplate (Submodel)

The `DigitalNameplate` submodel maps to a USD prim with `DppNameplateAPI` + `IndustrialEquipmentAPI` applied. The prim `kind` should be `component`.

Schema Definitions (illustrative)

```
# ── Full IDTA 02006-3-0 Digital Nameplate base ──────────────────────────────
class "DppNameplateAPI" (
    inherits = </APISchemaBase>
    doc = "Full IDTA 02006-3-0 Digital Nameplate base schema. For battery passport use with
BatteryNameplateAPI; for industrial equipment use with IndustrialEquipmentAPI."
    customData = {
        token apiSchemaType = "singleApply"
```

```
    }
) {
    # ── Mandatory identification ──────────────────────────────────────
    asset dpp:uriOfTheProduct (
        doc = "Globally unique URI for this product/passport instance. (AAS:
URIOfTheProduct, xs:anyURI, mandatory) Also written to sourceId:dpp:uriOfTheProduct."
    )
    string dpp:manufacturerName (
        doc = "Legally valid manufacturer name. (AAS: ManufacturerName, MLP, mandatory)"
    )

    # ── Optional product classification ──────────────────────────────
    string dpp:manufacturerProductDesignation (
        doc = "Short description or model name. (AAS: ManufacturerProductDesignation, MLP,
optional)"
    )
    string dpp:manufacturerProductRoot (
        doc = "Highest-level product grouping. (AAS: ManufacturerProductRoot, MLP,
optional)"
    )
    string dpp:manufacturerProductFamily (
        doc = "Product family designation. (AAS: ManufacturerProductFamily, MLP, optional)"
    )
    string dpp:manufacturerProductType (
        doc = "Product type or variant designation. (AAS: ManufacturerProductType, xs:string
Property, optional)"
    )
    string dpp:orderCode (
        doc = "Order code / catalog number. (AAS: OrderCodeOfManufacturer, xs:string
Property, mandatory) Also written to sourceId:dpp:orderCode."
    )
    string dpp:articleNumber (
        doc = "Product article number. (AAS: ProductArticleNumberOfManufacturer, MLP,
optional) Also written to sourceId:dpp:articleNumber."
    )

    # ── Instance and batch identifiers ────────────────────────────────
    string dpp:serialNumber (
        doc = "Per-instance serial number. (AAS: SerialNumber, xs:string)"
    )
    string dpp:batchNumber (
        doc = "Manufacturing batch or lot number. (AAS: BatchNumber, xs:string, optional)"
    )

    # ── Origin, dates, and facility ──────────────────────────────────
    string dpp:countryOfOrigin (
        doc = "ISO 3166-1 alpha-2 country of origin. (AAS: CountryOfOrigin, xs:string,
optional)"
    )
    string dpp:yearOfConstruction (
        doc = "Year of manufacture YYYY. (AAS: YearOfConstruction, xs:string, optional)"
    )
    string dpp:dateOfManufacture (
```

```
        doc = "Manufacturing date ISO 8601 YYYY-MM-DD. (AAS: DateOfManufacture, xs:date)"
    )
    string dpp:dateOfPuttingIntoService (
        doc = "Service start date ISO 8601 YYYY-MM-DD. Optional. (AAS:
DateOfPuttingIntoService)"
    )
    string dpp:uniqueFacilityIdentifier (
        doc = "Unique identifier for the manufacturing facility. (AAS:
UniqueFacilityIdentifier)"
    )

    # ── Version information ──────────────────────────────────────────
    string dpp:hardwareVersion (
        doc = "Hardware revision string. (AAS: HardwareVersion, xs:string Property,
optional)"
    )
    string dpp:firmwareVersion (
        doc = "Firmware revision string. (AAS: FirmwareVersion, xs:string Property,
optional)"
    )
    string dpp:softwareVersion (
        doc = "Software revision string. (AAS: SoftwareVersion, xs:string Property,
optional)"
    )

    # ── Media ────────────────────────────────────────────────────────
    asset dpp:companyLogo (
        doc = "Manufacturer company logo. (AAS: CompanyLogo, File, optional)"
    )

    # ── Contact information – flat encoding (Option 1) ──────────────
    string dpp:contact:street (
        doc = "Street address. (AAS: ContactInformation/Street, MLP)"
    )
    string dpp:contact:zipcode (
        doc = "Postal code. (AAS: ContactInformation/Zipcode, MLP)"
    )
    string dpp:contact:cityTown (
        doc = "City or town. (AAS: ContactInformation/CityTown, MLP)"
    )
    string dpp:contact:stateCounty (
        doc = "State, county, or province. (AAS: ContactInformation/StateCounty, MLP)"
    )
    string dpp:contact:nationalCode (
        doc = "ISO 3166-1 alpha-2 country for the contact address. (AAS:
ContactInformation/NationalCode, MLP)"
    )
    string dpp:contact:poBox (
        doc = "Post office box. (AAS: ContactInformation/POBox, MLP)"
    )
    string dpp:contact:company (
        doc = "Company name at contact address. (AAS: ContactInformation/Company, MLP)"
    )
```

```
    string dpp:contact:department (
        doc = "Department. (AAS: ContactInformation/Department, MLP)"
    )
    string dpp:contact:timeZone (
        doc = "IANA time zone. (AAS: ContactInformation/TimeZone, xs:string)"
    )
    string dpp:contact:phone (
        doc = "Primary telephone number. (AAS: ContactInformation/Phone/TelephoneNumber)"
    )
    string dpp:contact:fax (
        doc = "Primary fax number. (AAS: ContactInformation/Fax/FaxNumber)"
    )
    string dpp:contact:email (
        doc = "Primary email address. (AAS: ContactInformation/Email/EmailAddress)"
    )
    asset dpp:contact:additionalLink (
        doc = "Web address / URI. (AAS:
ContactInformation/IPCommunication/AddressOfAdditionalLink)"
    )
    string dpp:contact:nameOfContact (
        doc = "Family name of primary contact. (AAS: ContactInformation/NameOfContact, MLP)"
    )
    string dpp:contact:firstName (
        doc = "First name of primary contact. (AAS: ContactInformation/FirstName, MLP)"
    )
    string dpp:contact:remarks (
        doc = "Additional address remarks. (AAS: ContactInformation/AddressRemarks, MLP)"
    )
}

# ── Industrial equipment extension ──────────────────────────────────────
class "IndustrialEquipmentAPI" (
    inherits = </APISchemaBase>
    doc = "Industrial equipment extension for DppNameplateAPI. Signals full IDTA 02006-3-0
encoding including optional ExplosionSafeties and AssetSpecificProperties child prim
structures. Apply together with DppNameplateAPI for industrial equipment; do not apply for
battery passport prims."
    customData = {
        token apiSchemaType = "singleApply"
    }
) {
    # No top-level attributes. This schema is a discoverer: tooling can find all
    # fully-encoded IDTA 02006-3-0 industrial nameplate prims by checking
    # for IndustrialEquipmentAPI in applied schemas.
    # ExplosionSafetyAPI is applied to child prims, not to the equipment prim itself.
}

# ── Per-marking schema (shared with battery passport mapping) ──────────
class "MarkingAPI" (
    inherits = </APISchemaBase>
    doc = "Applied schema for a single Marking entry. Shared between DppNameplateAPI and
BatteryNameplateAPI workflows."
    customData = {
```

```
        token apiSchemaType = "singleApply"
    }
) {
    token marking:name (
        doc = "Marking type. (AAS: MarkingName, mandatory) Preferred: IRDI from IEC
CDD/ECLASS."
    )
    string marking:issueDate (
        doc = "Issue date ISO 8601 YYYY-MM-DD. (AAS: IssueDate, optional)"
    )
    string marking:expiryDate (
        doc = "Expiry date ISO 8601 YYYY-MM-DD. (AAS: ExpiryDate, optional)"
    )
    asset marking:file (
        doc = "Conformity symbol image. (AAS: MarkingFile, File, mandatory)"
    )
    string[] marking:additionalText (
        doc = "Additional explanatory text. (AAS: MarkingAdditionalText, SML of xs:string,
optional)"
    )
}

# ── Per-explosion-safety schema (industrial equipment only) ──────────────
class "ExplosionSafetyAPI" (
    inherits = </APISchemaBase>
    doc = "Applied to ExplosionSafety_NN child prims nested under
Markings/Marking_NN/ExplosionSafeties/. Encodes common fields of the IDTA 02006-3-0
ExplosionSafety SMC. Deep sub-SMCs (AmbientConditions, ProcessConditions,
ExternalElectricalCircuit) require further child prims for full fidelity."
    customData = {
        token apiSchemaType = "singleApply"
    }
) {
    string explosionSafety:typeOfApproval (
        doc = "Explosion protection approval type, e.g. ATEX, IECEx. (AAS: TypeOfApproval,
MLP)"
    )
    string explosionSafety:approvalAgency (
        doc = "Testing or certification agency. (AAS: ApprovalAgencyTestingInstitute, MLP)"
    )
    string explosionSafety:typeOfProtection (
        doc = "Protection concept, e.g. Ex d, Ex e, Ex ia. (AAS: TypeOfProtection, MLP)"
    )
    string explosionSafety:ratedInsulationVoltage (
        doc = "Rated insulation voltage with unit. (AAS: RatedInsulationVoltage, MLP)"
    )
    asset explosionSafety:instructionControlDrawing (
        doc = "Instruction or control drawing file. (AAS: InstructionControlDrawing, File)"
    )
    string explosionSafety:specificConditionsForUse (
        doc = "Conditions required for safe use. (AAS: SpecificConditionsForUse, MLP)"
    )
    string explosionSafety:incompleteDevice (
```

```
        doc = "Incomplete device indication per ATEX/IECEx. (AAS: IncompleteDevice, MLP)"
    )
}
```

AAS → USD

A `DigitalNameplate` Submodel instance is encoded as a USD prim with `DppNameplateAPI` + `IndustrialEquipmentAPI` applied.

AAS	OpenUSD	Description
Submodel semanticId	`customData["aas:submodelSemanticId"]` (string)	`https://admin-shell.io/idta/nameplate/3/0`

Usage Example

```
def Xform "Pump_MusterAG_CM5_3" (
    kind = "component"
    prepend apiSchemas = ["DppNameplateAPI", "IndustrialEquipmentAPI"]
    customData = {
        string "aas:submodelSemanticId" = "https://admin-shell.io/idta/nameplate/3/0"
        string "aas:semanticId:uriOfTheProduct"  = "0112/2///61987#ABN590#002"
        string "aas:semanticId:manufacturerName" = "0112/2///61987#ABA565#009"
        dictionary "dpp:manufacturerName:i18n" = {
            string "de" = "Muster AG"
            string "en" = "Muster AG"
        }
        dictionary "dpp:manufacturerProductDesignation:i18n" = {
            string "de" = "Kreiselpumpe"
            string "en" = "Centrifugal Pump"
        }
    }
    assetInfo = {
        asset identifier = @https://products.muster-ag.de/cm5-3/SN-20240001@
    }
) {
    # ── Source identifier entries ──────────────────────────────────
    custom string sourceId:dpp:uriOfTheProduct = "https://products.muster-ag.de/cm5-3/SN-
20240001"
    custom string sourceId:dpp:orderCode       = "96402234"
    custom string sourceId:dpp:articleNumber   = "CM5-3-I-A-E-AVBE"

    # ── DppNameplateAPI – identification ──────────────────────────
    asset  dpp:uriOfTheProduct              = @https://products.muster-ag.de/cm5-3/SN-
20240001@
    string dpp:manufacturerName             = "Muster AG"
    string dpp:manufacturerProductDesignation = "Centrifugal Pump"
    string dpp:manufacturerProductRoot      = "Pumps"
    string dpp:manufacturerProductFamily    = "CM"
    string dpp:manufacturerProductType      = "CM 5-3"
    string dpp:orderCode                    = "96402234"
    string dpp:articleNumber                = "CM5-3-I-A-E-AVBE"
```

```
    string dpp:serialNumber                 = "SN-20240001"
    string dpp:batchNumber                  = "BL-2024-Q1-042"
    string dpp:countryOfOrigin              = "DE"
    string dpp:yearOfConstruction           = "2024"
    string dpp:dateOfManufacture            = "2024-02-15"
    string dpp:uniqueFacilityIdentifier     = "GLN:4012345000016"
    string dpp:hardwareVersion              = "HW-Rev-C"
    string dpp:firmwareVersion              = "FW-2.4.1"
    string dpp:softwareVersion              = "SW-3.0.0"
    asset  dpp:companyLogo                  = @./media/muster_ag_logo.png@

    # ── DppNameplateAPI – contact information (Option 1, flat) ──────────────
    string dpp:contact:street       = "Musterstraße 1"
    string dpp:contact:zipcode      = "12345"
    string dpp:contact:cityTown     = "Musterstadt"
    string dpp:contact:nationalCode = "DE"
    string dpp:contact:company      = "Muster AG"
    string dpp:contact:phone        = "+49 123 456-0"
    string dpp:contact:email        = "contact@muster-ag.de"
    asset  dpp:contact:additionalLink = @https://www.muster-ag.de@

    # ── Markings ────────────────────────────────────────────────────────────
    def Scope "Markings" {
        def Scope "Marking_00" (prepend apiSchemas = ["MarkingAPI"]) {
            custom string sourceId:dpp:marking:certificationId = "BVS 21 ATEX E 001"
            token    marking:name                   = "0173-1#07-DAA603#004"
            string   marking:issueDate              = "2024-01-10"
            string   marking:expiryDate             = "2029-01-10"
            asset    marking:file                   = @./markings/ce_mark.png@
            string[] marking:additionalText         = ["Notified body: 0158"]

            def Scope "ExplosionSafeties" {
                def Scope "ExplosionSafety_00" (prepend apiSchemas = ["ExplosionSafetyAPI"])
{
                    string explosionSafety:typeOfApproval   = "ATEX"
                    string explosionSafety:approvalAgency   = "BVS Bochum"
                    string explosionSafety:typeOfProtection = "Ex d IIB T4"
                    asset  explosionSafety:instructionControlDrawing =
@./docs/atex_instruction.pdf@
                }
            }
        }
        def Scope "Marking_01" (prepend apiSchemas = ["MarkingAPI"]) {
            token marking:name = "WEEE"
            asset marking:file = @./markings/weee_mark.png@
        }
    }

    # ── AssetSpecificProperties ─────────────────────────────────────────────
    def Scope "AssetSpecificProperties" {
        def Scope "GuidelineProps_00" {
            string guideline:conformityDeclaration = "IEC 60034"
            customData = {
```

```
                dictionary "guideline:arbitraryProperties" = {
                    string "NominalPower"    = "1500 W"
                    string "NominalVoltage"  = "400 V"
                    string "ProtectionClass" = "IP55"
                }
            }
        }
    }
}
```

USD → AAS

1. **Identify the prim**: Must have `DppNameplateAPI` in applied schemas. Presence of `IndustrialEquipmentAPI` signals the full IDTA 02006-3-0 profile is intended.
2. **Construct Submodel envelope**: `idShort = "DigitalNameplate"`, `semanticId` from `customData["aas:submodelSemanticId"]` or the fixed value `https://admin-shell.io/idta/nameplate/3/0`.
3. **Populate SubmodelElements** per the per-field mappings below.
4. **ContactInformation**: read `dpp:contact:*` attrs (Option 1) or child prim `ContactInformation` (Option 2).
5. **Markings**: find child `Markings`; collect `MarkingAPI` children; sort by name to recover SML order. For each, check for `ExplosionSafeties` child scope and reconstruct.
6. **AssetSpecificProperties**: find child `AssetSpecificProperties`; collect `GuidelineProps_NN` children.
7. **AAS envelope** (Asset, AAS shell) must be supplied by the receiving system.

URIOfTheProduct

Globally unique URI identifying this product or product passport instance. Cross-system identifier — stored both as a source id entry (primary) and as a typed schema attribute.

AAS → USD

Written to three locations:

1. `sourceId:dpp:uriOfTheProduct` (string) — primary; verbatim URI, no SdfAssetPath resolution.
2. `dpp:uriOfTheProduct` (asset) — schema attribute; allows asset-path resolution by USD tools.
3. `assetInfo["identifier"]` — fallback for toolchains without source id support; model-root prims only.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`URIOfTheProduct`	`xs:anyURI`	—	Mandatory (1). semanticId: `0112/2///61987#ABN590#002`
OpenUSD (primary)	`sourceId:dpp:uriOfTheProduct`	—	`string`	Verbatim URI

	Name	AAS Type	USD Type	Notes
OpenUSD (schema attr)	`dpp:uriOfTheProduct`	—	`asset`	Schema discoverability
OpenUSD (fallback)	`assetInfo["identifier"]`	—	`asset`	Model roots only

USD → AAS

Read priority: `sourceId:dpp:uriOfTheProduct` → `dpp:uriOfTheProduct` → `assetInfo["identifier"]`. Write to `Property URIOfTheProduct` with `valueType = xs:anyURI`.

Round-trip: Lossless

ManufacturerName

Legally valid manufacturer name. AAS type MLP; primary string to attribute, language variants to `customData["dpp:manufacturerName:i18n"]`.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`ManufacturerName`	MLP	—	Mandatory (1). semanticId: `0112/2///61987#ABA565#009`
OpenUSD	`dpp:manufacturerName`	—	`string`	Primary language value

Round-trip: Lossy (language variants only; primary value lossless)

ManufacturerProductDesignation

Short description or model name of the product. Mandatory (1). MLP — same encoding as `ManufacturerName`; language variants to `customData["dpp:manufacturerProductDesignation:i18n"]`.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`ManufacturerProductDesignation`	MLP	—	Mandatory (1). semanticId: `0112/2///61987#ABA567#009`
OpenUSD	`dpp:manufacturerProductDesignation`	—	`string`	Primary language value

Round-trip: Lossy (language variants); absent field handled correctly

---

ContactInformation

The manufacturer's contact information, structured as an SMC using the IDTA 02002-1-0 Contact Information SMT. Mandatory in IDTA 02006-3-0 (cardinality 1).

The full IDTA 02002-1 structure includes deeply nested optional sub-SMCs (RoleOfContactPerson SML, Language SML, multiple Phone/Fax/Email SMCs). Both encoding options below are lossy for multi-valued and nested structures.

AAS → USD

Option 1 — Flat `dpp:contact:*` attributes (recommended for simple cases):

Each commonly used IDTA 02002-1 field maps to a `dpp:contact:<field>` attribute on the same prim. Covers the typical postal address and a primary contact point. Multi-entry structures (multiple phone numbers, roles) are truncated to the primary value.

Option 2 — Child `ContactInformation` scope prim:

A child `Scope` prim named `ContactInformation`. Attributes use bare names (without the `dpp:contact:` prefix). Recommended when contact data is shared across instances, when multi-entry sub-SMCs must be preserved, or when the full IDTA 02002-1 structure is required. For multiple phones use further child prims `Phone_NN`, `Fax_NN`, `Email_NN`.

Properties (commonly used IDTA 02002-1 fields)

AAS (ContactInformation)	USD Attribute (Option 1)	AAS Type	USD Type
Street	`dpp:contact:street`	MLP	`string`
Zipcode	`dpp:contact:zipcode`	MLP	`string`
CityTown	`dpp:contact:cityTown`	MLP	`string`
StateCounty	`dpp:contact:stateCounty`	MLP	`string`
NationalCode	`dpp:contact:nationalCode`	MLP	`string` (ISO 3166-1 alpha-2)
POBox	`dpp:contact:poBox`	MLP	`string`
Company	`dpp:contact:company`	MLP	`string`
Department	`dpp:contact:department`	MLP	`string`
TimeZone	`dpp:contact:timeZone`	`xs:string`	`string` (IANA TZ ID)
Phone / TelephoneNumber (primary)	`dpp:contact:phone`	MLP	`string`
Fax / FaxNumber (primary)	`dpp:contact:fax`	MLP	`string`

AAS (ContactInformation)	USD Attribute (Option 1)	AAS Type	USD Type
Email / EmailAddress (primary)	`dpp:contact:email`	`xs:string`	`string`
IPCommunication / AddressOfAdditionalLink	`dpp:contact:additionalLink`	`xs:anyURI`	`asset`
NameOfContact	`dpp:contact:nameOfContact`	MLP	`string`
FirstName	`dpp:contact:firstName`	MLP	`string`
AddressRemarks	`dpp:contact:remarks`	MLP	`string`

Option 2 Usage Example (multiple phones)

```
def Xform "Pump_MusterAG_CM5_3" (prepend apiSchemas = ["DppNameplateAPI",
"IndustrialEquipmentAPI"]) {
    def Scope "ContactInformation" {
        string street       = "Musterstraße 1"
        string zipcode      = "12345"
        string cityTown     = "Musterstadt"
        string nationalCode = "DE"
        string email        = "contact@muster-ag.de"
        asset  additionalLink = @https://www.muster-ag.de@
        def Scope "Phone_00" {
            string telephoneNumber = "+49 123 456-0"
            string typeOfTelephone = "0173-1#07-AAS754#001"
        }
        def Scope "Phone_01" {
            string telephoneNumber = "+49 123 456-999"
            string typeOfTelephone = "0173-1#07-AAS755#001"
        }
    }
}
```

USD → AAS

- **Option 1**: Read `dpp:contact:*` attrs; reconstruct `ContactInformation` SMC. Multi-entry sub-SMCs produce single entries.
- **Option 2**: Read child prim `ContactInformation`; read `Phone_NN` / `Fax_NN` / `Email_NN` children for multi-entry sub-SMCs.

Round-trip

Category	Fidelity
Postal address fields	Lossless
Primary phone / fax / email	Lossless
Multiple phone / fax / email	Lossless (Option 2 only)

Category	Fidelity
RoleOfContactPerson SML	Dropped (Option 1); child prim required (Option 2)
Language SML	Dropped in both options
MLP language variants	Lossy — `customData` convention required

ManufacturerProductRoot / ManufacturerProductFamily / ManufacturerProductType

Three-level product classification hierarchy defined by the manufacturer. All optional (0..1). `ManufacturerProductRoot` and `ManufacturerProductFamily` are MLP; `ManufacturerProductType` is a plain `xs:string` Property (not MLP) in the v3.0 template. MLP fields follow the same encoding pattern as `ManufacturerName`; language variants to `customData["dpp:<field>:i18n"]`. No i18n customData entry is needed for `ManufacturerProductType`.

AAS idShort	semanticId	USD Attribute
`ManufacturerProductRoot`	`0112/2///61987#ABA174#001`	`dpp:manufacturerProductRoot`
`ManufacturerProductFamily`	`0112/2///61987#ABA580#009`	`dpp:manufacturerProductFamily`
`ManufacturerProductType`	`0112/2///61987#ABA566#009`	`dpp:manufacturerProductType`

Round-trip: Lossy (language variants); absent fields handled correctly

---

OrderCodeOfManufacturer

The manufacturer's order code or catalog number. Mandatory (1). `xs:string` Property (not MLP) in the v3.0 template. The value is typically a cross-system catalog key appearing in PLM and procurement systems. Stored as both a source id entry and a schema attribute.

AAS → USD

1. `sourceId:dpp:orderCode` (string) — primary cross-system key, verbatim.
2. `dpp:orderCode` (string) — schema attribute, primary language value.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`OrderCodeOfManufacturer`	`xs:string`	—	Mandatory (1). semanticId: `0112/2///61987#ABA950#009`
OpenUSD (primary)	`sourceId:dpp:orderCode`	—	`string`	Verbatim cross-system key
OpenUSD (schema attr)	`dpp:orderCode`	—	`string`	Value

USD → AAS

Read `dpp:orderCode`; fallback `sourceId:dpp:orderCode`. Reconstruct as `xs:string` Property.

Round-trip: Lossy (language variants); value lossless via source id

---

ProductArticleNumberOfManufacturer

The manufacturer's product article number — the type-level identifier used in catalogs, PLM, and ERP systems. Same encoding strategy as `OrderCodeOfManufacturer`.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`ProductArticleNumberOfManufacturer`	MLP	—	Optional (0..1). semanticId: `0112/2///61987#ABA581#009`
OpenUSD (primary)	`sourceId:dpp:articleNumber`	—	`string`	Verbatim cross-system key
OpenUSD (schema attr)	`dpp:articleNumber`	—	`string`	Primary language value

Round-trip: Lossy (language variants); value lossless via source id

---

SerialNumber

Per-instance serial number. Optional in IDTA 02006-3-0 (0..1); mandatory in the battery passport context.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`SerialNumber`	`xs:string`	—	(0..1). semanticId: `0112/2///61987#ABA951#009`
OpenUSD	`dpp:serialNumber`	—	`string`	Not authored if absent

Round-trip: Lossless

BatchNumber

Manufacturing batch or lot number. Optional (0..1).

```
`BatchNumber` is not present in the IDTA 02006-3-0 v3.0 JSON template. It may be used as a pipeline extension or may appear in future revisions. The mapping below is included for completeness; verify against the official specification before depending on it.
```

Properties

	Name	AAS Type	USD Type	Notes
AAS	`BatchNumber`	`xs:string`	—	Optional (0..1). semanticId: `0112/2///61987#ABA952#009`
OpenUSD	`dpp:batchNumber`	—	`string`	Not authored if absent

Round-trip: Lossless

CountryOfOrigin

ISO 3166-1 alpha-2 country code for the country where the product was manufactured. Optional (0..1).

Properties

	Name	AAS Type	USD Type	Notes
AAS	`CountryOfOrigin`	`xs:string`	—	Optional (0..1)
OpenUSD	`dpp:countryOfOrigin`	—	`string`	Two-letter code, e.g. `"DE"`

Round-trip: Lossless

YearOfConstruction

Year of manufacture in YYYY format. Optional (0..1). Note: `DateOfManufacture` provides a more precise date when available; both can coexist.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`YearOfConstruction`	`xs:string`	—	Optional (0..1). semanticId: `0112/2///61987#ABE966#001`. Format: YYYY
OpenUSD	`dpp:yearOfConstruction`	—	`string`	YYYY string

Round-trip: Lossless (provided value is YYYY)

DateOfManufacture

Precise manufacturing date. USD has no native `xs:date`; ISO 8601 strings are used. Optional in the industrial nameplate context (0..1).

Properties

	Name	AAS Type	USD Type	Notes
AAS	`DateOfManufacture`	`xs:date`	—	(0..1). semanticId: `0112/2///61987#ABB757#007`
OpenUSD	`dpp:dateOfManufacture`	—	`string`	ISO 8601 YYYY-MM-DD

Round-trip: Lossless (provided string is valid ISO 8601)

UniqueFacilityIdentifier

Unique identifier for the manufacturing facility. Introduced in IDTA 02006-3-0 v3.0. Optional (0..1). This is a **location-scoped identifier**, distinct from the product URI and the manufacturer identifier.

Properties

	Name	AAS Type	USD Type	Notes
AAS	`UniqueFacilityIdentifier`	`xs:string`	—	Optional (0..1). semanticId: `https://admin-shell.io/idta/nameplate/3/0/UniqueFacilityIdentifier`
OpenUSD	`dpp:uniqueFacilityIdentifier`	—	`string`	Not authored if absent

Round-trip: Lossless

HardwareVersion / FirmwareVersion / SoftwareVersion

Hardware, firmware, and software revision strings. All `xs:string` Property (0..1) in the v3.0 template — not MLP. In practice version designations are language-neutral; no i18n customData is needed for these fields.

AAS idShort	USD Attribute
`HardwareVersion`	`dpp:hardwareVersion`
`FirmwareVersion`	`dpp:firmwareVersion`
`SoftwareVersion`	`dpp:softwareVersion`

Round-trip: Lossless

CompanyLogo

Image file of the manufacturer's company logo. AAS type `File` (binary asset with MIME type). Optional (0..1).

Properties

	Name	AAS Type	USD Type	Notes
AAS	`CompanyLogo`	`File`	—	Optional (0..1). semanticId: `0112/2///61987#ABA588#001`
OpenUSD	`dpp:companyLogo`	—	`asset`	File path or URI

The AAS `File` MIME type is not encoded in the USD `asset` type. If MIME type round-trip matters, preserve it in `customData["dpp:companyLogo:mimeType"]`.

Round-trip: Lossless for path; MIME type lossy unless preserved in `customData`

Markings

The `Markings` SML contains one or more `Marking` SMCs. In IDTA 02006-3-0, each `Marking` SMC may additionally contain an `ExplosionSafeties` SMC for explosion-protection approvals — an industrial equipment specific structure not present in the battery passport template.

Each marking → child prim `Marking_NN` with `MarkingAPI` applied. If `ExplosionSafeties` is present → nested `ExplosionSafeties` scope prim containing `ExplosionSafety_NN` child prims with `ExplosionSafetyAPI` applied.

Composition

```
Equipment  (Xform, DppNameplateAPI + IndustrialEquipmentAPI)
└── Markings  (Scope)
    ├── Marking_00  (Scope, MarkingAPI)
    |   └── ExplosionSafeties  (Scope)
    |       └── ExplosionSafety_00  (Scope, ExplosionSafetyAPI)
    └── Marking_01  (Scope, MarkingAPI)
```

AAS → USD

Each `Markings__NN__` SMC → `Marking_NN` (zero-padded index preserves SML order). `MarkingAPI` fields: same as in battery passport mapping. If the SMC contains `ExplosionSafeties`, create the nested prim hierarchy.

Properties (per Marking)

AAS	USD Attribute	AAS Type	USD Type	Notes
MarkingName	`marking:name`	`xs:string`	`token` (IRDI preferred)	Mandatory (1)
DesignationOfCertificateOrApproval	`sourceId:dpp:marking:certificationId`	`xs:string`	`string`	Optional (0..1)
IssueDate	`marking:issueDate`	`xs:date`	`string` ISO 8601	Optional (0..1)
ExpiryDate	`marking:expiryDate`	`xs:date`	`string` ISO 8601	Optional (0..1)
MarkingFile	`marking:file`	`File`	`asset`	Mandatory (1)
MarkingAdditionalText	`marking:additionalText`	SML of `xs:string`	`string[]`	Optional (0..*)

`sourceId:dpp:marking:certificationId` is the sole encoding for the certificate or approval number. Using a source id rather than a schema attribute avoids duplication and enables cross-system linking to approval databases (ATEX, IECEx, etc.) without requiring `MarkingAPI` schema knowledge. Omitted when no certificate designation is present.

Properties (per ExplosionSafety)

AAS	USD Attribute	AAS Type	USD Type
TypeOfApproval	`explosionSafety:typeOfApproval`	MLP	`string`
ApprovalAgencyTestingInstitute	`explosionSafety:approvalAgency`	MLP	`string`
TypeOfProtection	`explosionSafety:typeOfProtection`	MLP	`string`
RatedInsulationVoltage	`explosionSafety:ratedInsulationVoltage`	MLP	`string`
InstructionControlDrawing	`explosionSafety:instructionControlDrawing`	`File`	`asset`
SpecificConditionsForUse	`explosionSafety:specificConditionsForUse`	MLP	`string`
IncompleteDevice	`explosionSafety:incompleteDevice`	MLP	`string`
AmbientConditions (sub-SMC)	child prim `AmbientConditions`	SMC	custom attrs
ProcessConditions (sub-SMC)	child prim `ProcessConditions`	SMC	custom attrs
ExternalElectricalCircuit (0..*)	child prims `ExternalElectricalCircuit_NN`	SML of SMC	custom attrs

`AmbientConditions`, `ProcessConditions`, and `ExternalElectricalCircuit` are deeply nested and domain-specific. When full fidelity is required, encode each field as a custom string or numeric attribute on dedicated child scope prims using the AAS idShort names.

USD → AAS

1. Find child prim `Markings`.
2. Collect children with `MarkingAPI`; sort by name to recover SML order.
3. For each, construct `Markings__NN__` SMC.
4. If a `ExplosionSafeties` child scope exists under the marking, collect its `ExplosionSafetyAPI` children; sort by name; reconstruct the `ExplosionSafeties` SMC.

Round-trip

Field	Fidelity
All MarkingAPI fields	Lossless
SML ordering	Lossless (requires `Marking_NN` naming convention)
All ExplosionSafetyAPI fields	Lossless
AmbientConditions / ProcessConditions	Partial — only authored custom attributes
ExternalElectricalCircuit	Partial — only authored custom attributes

---

AssetSpecificProperties

An optional SMC (0..1) providing application-specific properties not covered by the standard nameplate. Contains one or more `GuidelineSpecificProperties__NN__` SMCs grouping properties under a specific standard or guideline.

Structure

```
AssetSpecificProperties (SMC)
└── GuidelineSpecificProperties__00__ (SMC)
    ├── GuidelineForConformityDeclaration (Property, xs:string, mandatory)
    └── ArbitraryPropertiesOfAssetTypeValuePairs (SML of SMC, optional)
        └── [each SMC]: PropertyIdShort, PropertyClassificationSystem,
                        PropertySemanticId, PropertyName (MLP), PropertyValue (MLP),
PropertyUnit (MLP)
```

AAS → USD

Each `GuidelineSpecificProperties__NN__` SMC → child scope prim `GuidelineProps_NN` under an `AssetSpecificProperties` scope prim. `GuidelineForConformityDeclaration` → `guideline:conformityDeclaration` string attribute. Arbitrary properties → `customData["guideline:arbitraryProperties"]` dictionary keyed by property idShort with string values. For full metadata fidelity (units, semanticIds, MLP name/value), use nested child prims.

```
def Scope "AssetSpecificProperties" {
    def Scope "GuidelineProps_00" {
        string guideline:conformityDeclaration = "IEC 60034"
        customData = {
            string "aas:semanticId" = "0112/2///61987#ABN492#001"
            dictionary "guideline:arbitraryProperties" = {
                string "NominalPower"    = "1500 W"
                string "NominalVoltage"  = "400 V"
                string "ProtectionClass" = "IP55"
            }
        }
    }
    def Scope "GuidelineProps_01" {
        string guideline:conformityDeclaration = "EN 13849"
        customData = {
            dictionary "guideline:arbitraryProperties" = {
                string "PerformanceLevel" = "PLd"
                string "Category"         = "3"
            }
        }
    }
}
```

USD → AAS

1. Find child prim `AssetSpecificProperties`.
2. Collect `GuidelineProps_NN` children; sort by name.
3. For each: `guideline:conformityDeclaration` → `GuidelineForConformityDeclaration`.
4. `customData["guideline:arbitraryProperties"]` dictionary → `ArbitraryPropertiesOfAssetTypeValuePairs` SML; each key-value pair → one SMC with `PropertyIdShort` = key and `PropertyValue` = value.

Round-trip

Field	Fidelity
GuidelineForConformityDeclaration	Lossless
Arbitrary property values	Lossless (stored as strings)
PropertyClassificationSystem, PropertySemanticId	Dropped (not in `customData` dict encoding)
PropertyName, PropertyUnit (MLP)	Dropped (not in `customData` dict encoding)

For full metadata fidelity of the arbitrary properties, use nested child prims instead of the `customData` dictionary approach.

USD-Native Concepts

These USD concepts have no AAS equivalent and are dropped on USD → AAS export.

Composition Arcs

`references`, `payloads`, `inherits`, `specializes`, `over`. Flatten the composed prim before reading attribute values.

Time-Varying Attributes

Use `UsdAttribute::Get()` with `UsdTimeCode::Default()`. Fall back to `UsdTimeCode(0)`.

Variant Sets

Export the currently selected variant only.

Relationships

No Digital Nameplate SubmodelElement is relationship-typed. Drop on export.

Appendices

Appendix A: Round-trip Fidelity Summary

AAS Field	Direction	Fidelity	Loss / Condition
URIOfTheProduct	Both	Lossless	Primary: `sourceId:dpp:uriOfTheProduct`; fallback: `dpp:uriOfTheProduct` → `assetInfo["identifier"]`
ManufacturerName (primary)	Both	Lossless	—
ManufacturerName (language variants)	Both	Lossy	`customData` convention required
ManufacturerProductDesignation	Both	Lossy	Primary language only
ContactInformation (postal + single contact)	Both	Lossless	—
ContactInformation (multiple phone/fax/email)	Both	Lossless	Option 2 only; Option 1 truncates
ContactInformation (RoleOfContactPerson, Language SML)	Both	Dropped	Not covered by either option
ManufacturerProductRoot/Family/Type	Both	Lossy	Primary language only
OrderCodeOfManufacturer (value)	Both	Lossless	Via `sourceId:dpp:orderCode`
OrderCodeOfManufacturer (language variants)	Both	Lossy	`customData` only
ProductArticleNumberOfManufacturer (value)	Both	Lossless	Via `sourceId:dpp:articleNumber`
SerialNumber	Both	Lossless	—
BatchNumber	Both	Lossless	—
CountryOfOrigin	Both	Lossless	—
YearOfConstruction	Both	Lossless	Must be YYYY
DateOfManufacture	Both	Lossless	Must be valid ISO 8601
UniqueFacilityIdentifier	Both	Lossless	—
HardwareVersion / FirmwareVersion / SoftwareVersion	Both	Lossy	Primary language only (practically lossless)
CompanyLogo (path)	Both	Lossless	—
CompanyLogo (MIME type)	Both	Lossy	Use `customData["dpp:companyLogo:mimeType"]`
Markings (all MarkingAPI fields)	Both	Lossless	—
Markings (SML ordering)	Both	Lossless	Requires `Marking_NN` convention
ExplosionSafeties (all ExplosionSafetyAPI fields)	Both	Lossless	—
AmbientConditions / ProcessConditions	Both	Partial	Only authored custom attributes
AssetSpecificProperties (guideline + values)	Both	Lossless	Property metadata (unit, semanticId) dropped
semanticIds	Both	Conditional	Survive only if `customData` not stripped
AAS Asset / AAS hierarchy	AAS → USD	Dropped	Not encoded in USD
Composition arcs	USD → AAS	Dropped	Flatten before export

AAS Field	Direction	Fidelity	Loss / Condition
Time-varying attributes	USD → AAS	Dropped	Use default value
Variant sets	USD → AAS	Dropped	Export selected variant

Appendix B: Identifier Encoding Strategies

AAS Field	Identifier Scope	Primary USD Encoding	Notes
`URIOfTheProduct`	Per-instance product URI	`sourceId:dpp:uriOfTheProduct` (string)	Fallback: `dpp:uriOfTheProduct` (asset) → `assetInfo["identifier"]`
`OrderCodeOfManufacturer`	Manufacturer catalog / type-level	`sourceId:dpp:orderCode` (string)	`dpp:orderCode` (string) also written
`ProductArticleNumberOfManufacturer`	PLM / ERP type-level key	`sourceId:dpp:articleNumber` (string)	`dpp:articleNumber` (string) also written
`SerialNumber`	Per-instance	`dpp:serialNumber` (string)	No source id required by default
`BatchNumber`	Per-batch	`dpp:batchNumber` (string)	No source id required by default
`UniqueFacilityIdentifier`	Location / facility	`dpp:uniqueFacilityIdentifier` (string)	No source id required by default

`URIOfTheProduct` must survive verbatim — SdfAssetPath resolution semantics would corrupt a product passport URI. `OrderCodeOfManufacturer` and `ProductArticleNumberOfManufacturer` are type-level keys appearing as foreign keys in PLM and procurement systems; source id entries make them discoverable without loading the `dpp:` schema.

`SerialNumber`, `BatchNumber`, and `UniqueFacilityIdentifier` do not require source id entries by default. If they appear as foreign keys in an external system and cross-system linking is required, add `sourceId:dpp:<field>` as a pipeline extension.

Alternative: `assetInfo` sub-dictionary (Approach A)

```
assetInfo = {
    dictionary sourceIdentifiers = {
        dictionary dpp = {
            string uriOfTheProduct = "https://products.muster-ag.de/cm5-3/SN-20240001"
            string orderCode       = "96402234"
            string articleNumber   = "CM5-3-I-A-E-AVBE"
        }
    }
}
```

Appendix C: AAS-to-USD Type Mapping

AAS Type	USD Type	AAS → USD	USD → AAS	Fidelity
`xs:string`	`string`	Direct	Direct	Lossless

AAS Type	USD Type	AAS → USD	USD → AAS	Fidelity
`xs:anyURI`	`string` (source id) + `asset` (attr)	URI as source id + asset	Source id → URI	Lossless
`xs:date`	`string`	ISO 8601 string	Parse ISO 8601	Lossless if valid
MLP (primary)	`string`	Primary string	Single-entry MLP	Lossless
MLP (all variants)	`string` + `customData` dict	Primary + i18n dict	Reconstruct from dict	Lossy if `customData` stripped
`File`	`asset`	File path	Asset path	Lossless for path
`SMC` (flat)	Namespaced attrs	`ns:field`	Read `ns:field`	Lossless
`SML` (ordered, complex)	Child prims `Scope_NN`	Index-named child prims	Sort by name	Lossless if naming convention respected
`SML` (ordered, scalars)	Array attr	`type[]`	Array → list of Properties	Lossless

Appendix D: Relationship to Battery Passport (DppNameplateAPI + BatteryNameplateAPI)

`DppNameplateAPI` is shared between the industrial nameplate and the battery passport workflows. The table below shows which fields are used in each context.

`DppNameplateAPI` field	Industrial nameplate	Battery passport	Notes
`dpp:uriOfTheProduct`	✓ mandatory	✓ mandatory	—
`dpp:manufacturerName`	✓ mandatory	✓ mandatory	—
`dpp:manufacturerProductDesignation`	✓ mandatory	—	—
`dpp:manufacturerProductRoot/Family/Type`	✓ optional	—	Industrial only
`dpp:orderCode`	✓ mandatory	—	—
`dpp:articleNumber`	✓ optional	—	Industrial only
`dpp:serialNumber`	✓ optional	✓ mandatory	Cardinality differs by domain
`dpp:batchNumber`	✓ optional	—	Not in v3.0 template; pipeline extension
`dpp:countryOfOrigin`	✓ optional	—	—
`dpp:yearOfConstruction`	✓ optional	—	Industrial only
`dpp:dateOfManufacture`	✓ optional	✓ mandatory	—
`dpp:dateOfPuttingIntoService`	—	✓ optional	Battery passport specific
`dpp:uniqueFacilityIdentifier`	✓ optional	✓ mandatory	—
`dpp:hardwareVersion/firmwareVersion/softwareVersion`	✓ optional	—	Industrial only
`dpp:companyLogo`	✓ optional	—	Industrial only
`dpp:contact:*`	✓ full set	✓ mandatory subset	Battery passport requires street/zipcode/cityTown/nationalCode

`DppNameplateAPI` field	Industrial nameplate	Battery passport	Notes
`Markings/Marking_NN` (`MarkingAPI`)	✓ optional + `ExplosionSafeties`	✓ optional	`ExplosionSafeties` industrial only
`AssetSpecificProperties`	✓ optional	—	Industrial only (via `IndustrialEquipmentAPI`)

Schema applied to the prim:

Use case	Applied schemas
Industrial equipment nameplate (IDTA 02006-3-0 full)	`DppNameplateAPI` + `IndustrialEquipmentAPI`
Battery passport (IDTA 02035-1)	`DppNameplateAPI` + `BatteryNameplateAPI`
Partial nameplate (no domain extension)	`DppNameplateAPI` only

Do **not** apply both `IndustrialEquipmentAPI` and `BatteryNameplateAPI` to the same prim; these represent mutually exclusive domain profiles of the same base template.